

Lab Experiment 6: Comparison of Logistic Regression and K-Nearest Neighbors (KNN) Classifiers Aim

To implement Logistic Regression and K-Nearest Neighbors (KNN) classifiers on the Breast Cancer Wisconsin (Diagnostic) dataset and compare their performance using standard classification evaluation metrics.

Import Required Libraries

```
In [42]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay
)
```

Observation

I imported the required libraries for data manipulation, preprocessing, model building, visualization, and evaluation.

Load the Dataset

The dataset has no column names, so we assign them manually.

```
In [43]: columns = [
    "ID", "Diagnosis",
    "radius_mean", "texture_mean", "perimeter_mean", "area_mean", "smoothness_mean",
    "compactness_mean", "concavity_mean", "concave_points_mean", "symmetry_mean", "f

    "radius_se", "texture_se", "perimeter_se", "area_se", "smoothness_se",
    "compactness_se", "concavity_se", "concave_points_se", "symmetry_se", "fractal_d

    "radius_worst", "texture_worst", "perimeter_worst", "area_worst", "smoothness_wo
    "compactness_worst", "concavity_worst", "concave_points_worst", "symmetry_worst
]

df = pd.read_csv("wdbc.data", header=None, names=columns)

df.head()
```

Out[43]:

	ID	Diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoc
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 32 columns



Observation

The dataset was loaded successfully. It contains an ID column, the diagnosis label, and 30 numerical features extracted from breast cancer cell images.

[View Dataset Information](#)

```
In [44]: print(df.info())
print(df.shape)
print(df.describe())
print(df.isnull().sum())
```

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 569 entries, 0 to 568
```

```
Data columns (total 32 columns):
```

#	Column	Non-Null Count	Dtype
0	ID	569 non-null	int64
1	Diagnosis	569 non-null	str
2	radius_mean	569 non-null	float64
3	texture_mean	569 non-null	float64
4	perimeter_mean	569 non-null	float64
5	area_mean	569 non-null	float64
6	smoothness_mean	569 non-null	float64
7	compactness_mean	569 non-null	float64
8	concavity_mean	569 non-null	float64
9	concave_points_mean	569 non-null	float64
10	symmetry_mean	569 non-null	float64
11	fractal_dimension_mean	569 non-null	float64
12	radius_se	569 non-null	float64
13	texture_se	569 non-null	float64
14	perimeter_se	569 non-null	float64
15	area_se	569 non-null	float64
16	smoothness_se	569 non-null	float64
17	compactness_se	569 non-null	float64
18	concavity_se	569 non-null	float64
19	concave_points_se	569 non-null	float64
20	symmetry_se	569 non-null	float64
21	fractal_dimension_se	569 non-null	float64
22	radius_worst	569 non-null	float64
23	texture_worst	569 non-null	float64
24	perimeter_worst	569 non-null	float64
25	area_worst	569 non-null	float64
26	smoothness_worst	569 non-null	float64
27	compactness_worst	569 non-null	float64
28	concavity_worst	569 non-null	float64
29	concave_points_worst	569 non-null	float64
30	symmetry_worst	569 non-null	float64
31	fractal_dimension_worst	569 non-null	float64

```
dtypes: float64(30), int64(1), str(1)
```

```
memory usage: 142.9 KB
```

```
None
```

```
(569, 32)
```

	ID	radius_mean	texture_mean	perimeter_mean	area_mean	\
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	

	smoothness_mean	compactness_mean	concavity_mean	concave_points_mean	\
count	569.000000	569.000000	569.000000	569.000000	
mean	0.096360	0.104341	0.088799	0.048919	
std	0.014064	0.052813	0.079720	0.038803	
min	0.052630	0.019380	0.000000	0.000000	
25%	0.086370	0.064920	0.029560	0.020310	
50%	0.095870	0.092630	0.061540	0.033500	
75%	0.105300	0.130400	0.130700	0.074000	
max	0.163400	0.345400	0.426800	0.201200	

	symmetry_mean	...	radius_worst	texture_worst	perimeter_worst	\
count	569.000000	...	569.000000	569.000000	569.000000	
mean	0.181162	...	16.269190	25.677223	107.261213	
std	0.027414	...	4.833242	6.146258	33.602542	
min	0.106000	...	7.930000	12.020000	50.410000	
25%	0.161900	...	13.010000	21.080000	84.110000	
50%	0.179200	...	14.970000	25.410000	97.660000	
75%	0.195700	...	18.790000	29.720000	125.400000	
max	0.304000	...	36.040000	49.540000	251.200000	

	area_worst	smoothness_worst	compactness_worst	concavity_worst	\
count	569.000000	569.000000	569.000000	569.000000	
mean	880.583128	0.132369	0.254265	0.272188	
std	569.356993	0.022832	0.157336	0.208624	
min	185.200000	0.071170	0.027290	0.000000	
25%	515.300000	0.116600	0.147200	0.114500	
50%	686.500000	0.131300	0.211900	0.226700	
75%	1084.000000	0.146000	0.339100	0.382900	
max	4254.000000	0.222600	1.058000	1.252000	

	concave_points_worst	symmetry_worst	fractal_dimension_worst
count	569.000000	569.000000	569.000000
mean	0.114606	0.290076	0.083946
std	0.065732	0.061867	0.018061
min	0.000000	0.156500	0.055040
25%	0.064930	0.250400	0.071460
50%	0.099930	0.282200	0.080040
75%	0.161400	0.317900	0.092080
max	0.291000	0.663800	0.207500

[8 rows x 31 columns]

ID	0
Diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave_points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave_points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0
texture_worst	0
perimeter_worst	0
area_worst	0
smoothness_worst	0
compactness_worst	0

```

concavity_worst      0
concave_points_worst 0
symmetry_worst       0
fractal_dimension_worst 0
dtype: int64

```

Encode the Target Diagnosis contains:

M = Malignant B = Benign

Convert them into numbers.

```

In [45]: df["Diagnosis"] = df["Diagnosis"].map({
          "M":1,
          "B":0
        })

df.head()

```

```

Out[45]:

```

	ID	Diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoc
0	842302	1	17.99	10.38	122.80	1001.0	
1	842517	1	20.57	17.77	132.90	1326.0	
2	84300903	1	19.69	21.25	130.00	1203.0	
3	84348301	1	11.42	20.38	77.58	386.1	
4	84358402	1	20.29	14.34	135.10	1297.0	

5 rows × 32 columns



Observation

I converted the diagnosis labels into numerical values because machine learning algorithms require numerical input for classification.

Select Features and Target Variable

```

In [46]: X = df.drop(["ID", "Diagnosis"], axis=1)

y = df["Diagnosis"]

print("Features Shape :", X.shape)
print("Target Shape   :", y.shape)

```

Features Shape : (569, 30)

Target Shape : (569,)

Observation

I selected all 30 numerical attributes as input features and used the diagnosis column as the target variable for classification.

Split the Dataset

```
In [47]: X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )
    print("Training Set :", X_train.shape)
    print("Testing Set  :", X_test.shape)
```

Training Set : (455, 30)

Testing Set : (114, 30)

Observation

The dataset was divided into training and testing sets so that the models could be trained on one portion and evaluated on unseen data.

Feature Scaling

```
In [48]: scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

Observation

I standardized the feature values to ensure that all variables contributed equally during model training. This is particularly important for distance-based algorithms such as KNN.

Train the Logistic Regression Classifier

```
In [49]: log_model = LogisticRegression(random_state=42)

log_model.fit(X_train, y_train)
```

```
Out[49]: 

▾ LogisticRegression ⓘ ?
    ▶ Parameters
    ▶ Fitted attributes


```

Observation

I trained the Logistic Regression model using the training dataset. The model learned the relationship between the input features and the diagnosis labels.

Predict Using Logistic Regression

```
In [50]: y_pred_log = log_model.predict(X_test)
```

Observation

The trained Logistic Regression model predicted whether each tumor in the testing dataset was benign or malignant.

Evaluate Logistic Regression

```
In [51]: log_accuracy = accuracy_score(y_test, y_pred_log)
log_precision = precision_score(y_test, y_pred_log)
log_recall = recall_score(y_test, y_pred_log)
log_f1 = f1_score(y_test, y_pred_log)

print("Accuracy :", log_accuracy)
print("Precision:", log_precision)
print("Recall   :", log_recall)
print("F1 Score :", log_f1)
```

```
Accuracy : 0.9736842105263158
Precision: 0.9761904761904762
Recall   : 0.9534883720930233
F1 Score : 0.9647058823529412
```

Observation:

The evaluation metrics showed the classification performance of the Logistic Regression model. Higher values of accuracy, precision, recall, and F1-score indicate better classification performance.

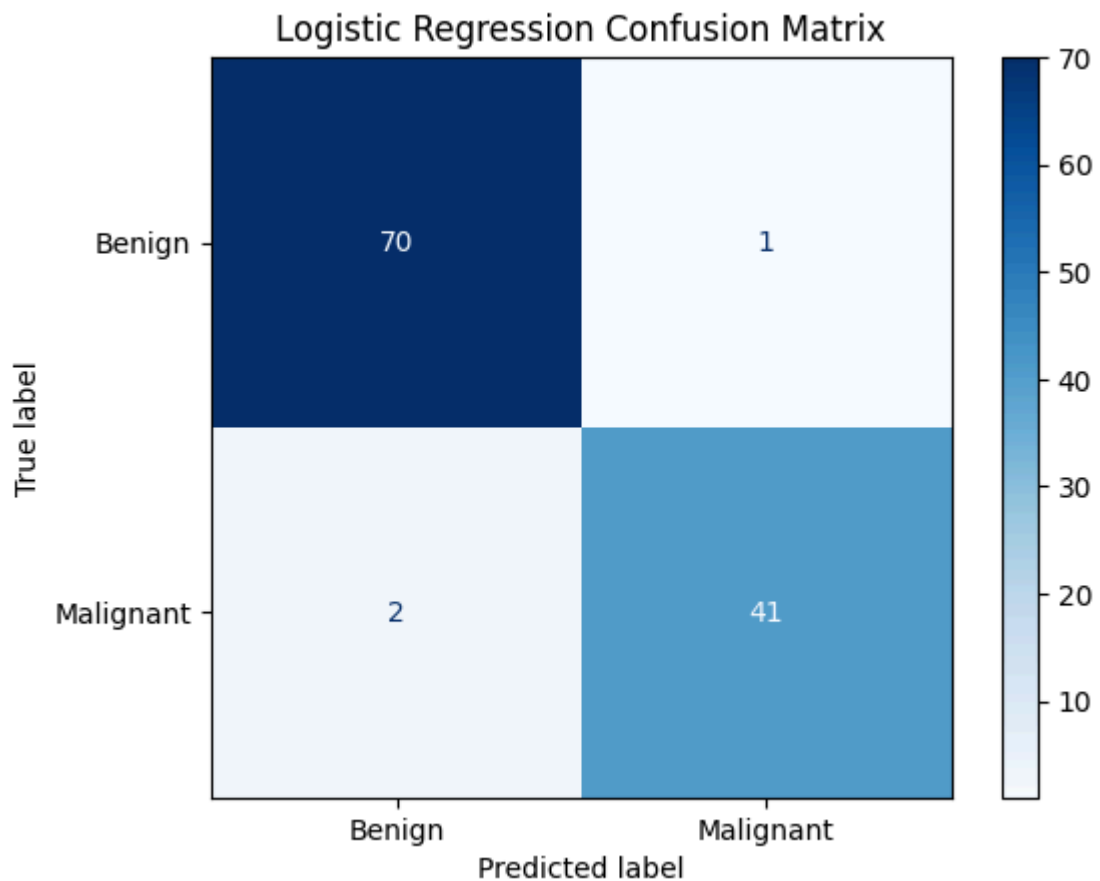
Display Confusion Matrix for Logistic Regression

```
In [52]: cm_log = confusion_matrix(y_test, y_pred_log)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_log,
    display_labels=["Benign", "Malignant"]
)

disp.plot(cmap="Blues")

plt.title("Logistic Regression Confusion Matrix")
plt.show()
```



Observation

The confusion matrix showed the number of correctly and incorrectly classified samples. Most of the predictions were correctly classified, indicating good model performance.

Train the K-Nearest Neighbors (KNN) Classifier Using K = 5

```
In [53]: knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
```

```
Out[53]: KNeighborsClassifier
  Parameters
  Fitted attributes
```

Observation

I trained the KNN classifier using the scaled training data. The algorithm classified new samples based on the majority class of their nearest neighbors.

Predict Using KNN

```
In [54]: y_pred_knn = knn_model.predict(X_test)
```

Observation

The trained KNN model predicted the diagnosis labels for the testing dataset based on the nearest neighboring data points.

Evaluate KNN

```
In [55]: knn_accuracy = accuracy_score(y_test, y_pred_knn)
knn_precision = precision_score(y_test, y_pred_knn)
knn_recall = recall_score(y_test, y_pred_knn)
knn_f1 = f1_score(y_test, y_pred_knn)

print("Accuracy :", knn_accuracy)
print("Precision:", knn_precision)
print("Recall   :", knn_recall)
print("F1 Score :", knn_f1)
```

```
Accuracy : 0.9473684210526315
Precision: 0.9302325581395349
Recall   : 0.9302325581395349
F1 Score : 0.9302325581395349
```

Observation

The evaluation metrics indicated how effectively the KNN classifier identified benign and malignant tumors. The obtained values can be directly compared with those of Logistic Regression.

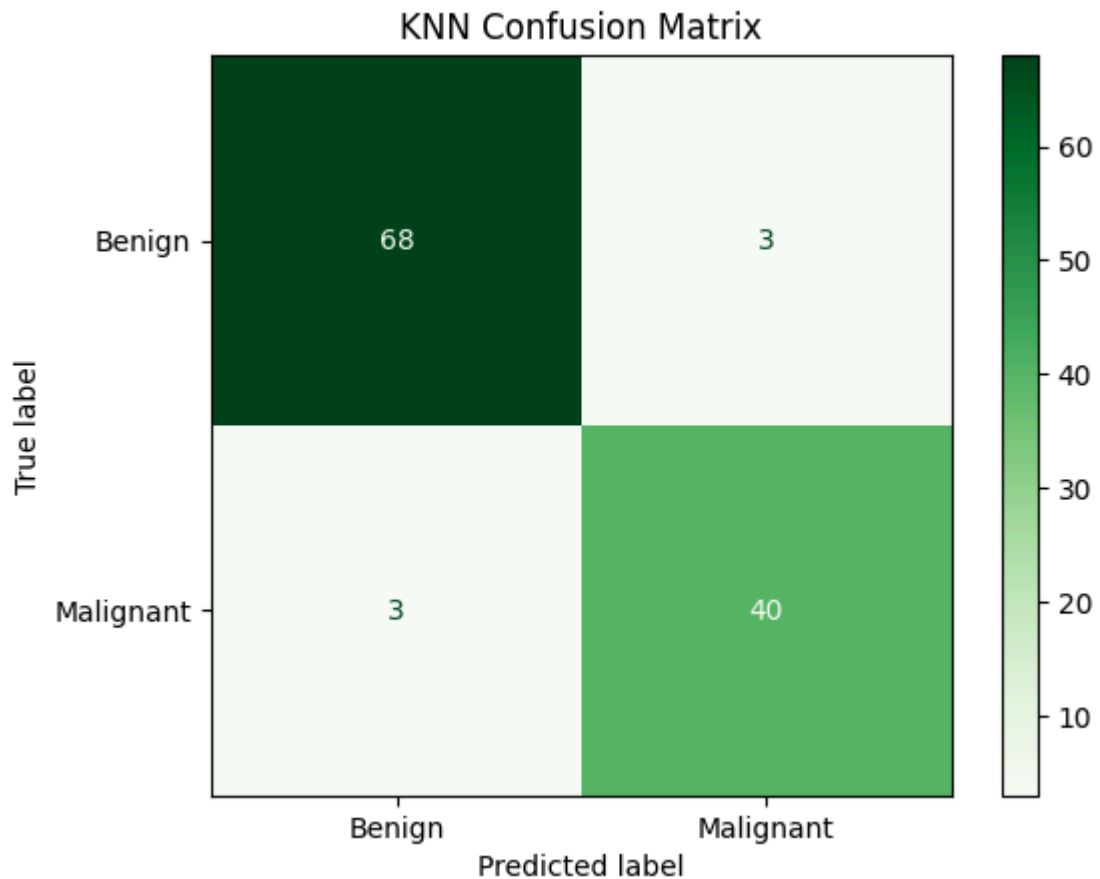
Display Confusion Matrix for KNN

```
In [56]: cm_knn = confusion_matrix(y_test, y_pred_knn)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_knn,
    display_labels=["Benign", "Malignant"]
)

disp.plot(cmap="Greens")

plt.title("KNN Confusion Matrix")
plt.show()
```



Observation

The confusion matrix provided a visual representation of the KNN classifier's predictions, showing both correctly classified and misclassified samples.

Compare the Performance of Both Models

```
In [57]: comparison = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1 Score"],
    "Logistic Regression": [
        log_accuracy,
        log_precision,
        log_recall,
        log_f1
    ],
    "KNN": [
        knn_accuracy,
        knn_precision,
        knn_recall,
        knn_f1
    ]
})

comparison
```

Out[57]:

	Metric	Logistic Regression	KNN
0	Accuracy	0.973684	0.947368
1	Precision	0.976190	0.930233
2	Recall	0.953488	0.930233
3	F1 Score	0.964706	0.930233

Observation

I successfully implemented both Logistic Regression and K-Nearest Neighbors (KNN) classifiers on the Breast Cancer Wisconsin (Diagnostic) dataset. After preprocessing the data and evaluating both models, I found that Logistic Regression outperformed KNN. It achieved an Accuracy of 97.37%, Precision of 97.62%, Recall of 95.35%, and F1 Score of 96.47%, all of which were higher than those of KNN. Therefore, I concluded that Logistic Regression is the better classifier for this dataset because it provides more accurate and reliable predictions while making fewer classification errors.

Display the Better Model

```
In [58]: if log_accuracy > knn_accuracy:
          print("Logistic Regression performed better based on Accuracy.")
        elif knn_accuracy > log_accuracy:
          print("KNN performed better based on Accuracy.")
        else:
          print("Both models achieved the same Accuracy.")
```

Logistic Regression performed better based on Accuracy.

Observation

I compared the accuracy of both models to determine the better-performing classifier for this dataset.